Distributed Multi-Model Analytics for E-commerce Data

Technical project report

Project scope	Implemented evidence
MongoDB documents and analytics	Loaded into local MongoDB; both aggregation pipelines executed and results saved.
HBase wide-column model	Started local HBase 2.6.6 and Thrift; created tables and loaded 99,873 page-view rows.
Spark processing and integration	Processed all 2,000,000 sessions and 500,000 transactions with Spark summaries; separately joined MongoDB and HBase sources on the assignment sample.
Visual analytics	Four static charts from the assignment-scale sample; a synthetic Spark stream also produced windowed microbatch aggregates.

Scale distinction: the full generator produced 10,000 users, 5,000 products, 25 categories, 2,000,000 sessions and 500,000 transactions, and Spark batch analysis ran over those files. MongoDB and HBase plus the direct cross-store integration use the manageable assignment sample: 1,000 users, 500 products, 10,000 sessions and 2,500 transactions.

1. Architecture and data

The design assigns each workload to the storage or processing model that best fits its access pattern. MongoDB holds catalog, profile, session-summary and transaction documents. HBase stores high-volume page-view events for keyed time-range retrieval. Spark batch analytics reads the generated JSON files. A separate integration mode reads profiles and fulfilled transactions from MongoDB through PyMongo and event rows from HBase through HappyBase, stages those extracts as JSON, then joins and aggregates them with Spark DataFrames. The store integration uses the assignment-scale load; full-scale batch processing uses the generated full dataset.

Stage	Technology	Purpose
Source	JSON files	Synthetic users, products, categories, sessions and transactions.
Operational documents	MongoDB	Nested records, selective indexes, lookup and group pipelines.
Event retrieval	HBase	Sparse page events keyed for a user-first recent-activity scan.
Analytics	Spark	Distributed joins, cohort aggregation, co-purchase and integrated metrics.
Presentation	Matplotlib + PDF	Four static plots and findings for a reviewable report.

Data preparation

The supplied dataset_generator.py is an exported notebook JSON object, not an executable Python module. run_assignment_generator.py extracts its code cell, overrides the scale values, and runs that code unchanged. The assignment-scale store sample contains 1,000 users, 500 products, 10,000 sessions and 2,500 transactions. The same runner generated the brief defaults: 10,000 users, 5,000 products, 25 categories, 500,000 transactions and 2,000,000 sessions, written as bounded chunks. Spark batch analysis was executed on the full files; MongoDB and HBase intentionally retain the representative sample. A separate standard-library sample generator remains available for quick smoke runs.

Pipeline sequence

Generate JSON -> MongoDB and HBase loaders (optional) -> Spark normalization and analysis -> Parquet/CSV aggregates plus PNG charts -> report. On a cluster, keep joins distributed, partition on user or event date where appropriate, and write compact columnar outputs for repeat analysis.

2. MongoDB and HBase design

MongoDB collections

Collection	Document boundary and index rationale
products	One document per product. Keep price history bounded and category/subcategory IDs for joins. Index product_id uniquely and category_id + is_active for catalog filtering.
users	One document per user with nested geo and registration fields. Keep large session histories out of this document. Index user_id uniquely.
transactions	One document per order, with its bounded items array embedded. Index transaction_id, timestamp + status, and user_id + timestamp.
sessions	One document per session with nested page_views, device and cart. This keeps the journey readable as one unit. Index user_id + start_time descending.

The implemented product-popularity pipeline filters fulfilled orders, unwinds items, groups units and item revenue, looks up product names and ranks the top 20. The category-revenue pipeline unwinds items, joins products and groups revenue, units and distinct orders by category. Use allowDiskUse for larger group stages. For repeatable imports, productionize replacement with bulk upserts and validate duplicate identifiers before loading.

HBase tables

Table	Row key	Column families
user_events	user_id | reverse_epoch_ms | session_id | event_index	s: session attributes; e: page type, product, category, duration, event timestamp
product_daily	product_id | yyyyMMdd	m: product views, cart-add units, purchased units, revenue in cents

The user_events key clusters event rows for each user and sorts newest first because the epoch is reversed. It supports a contiguous row-key range scan for one user. The product_daily table supports a product and date range query. Prefixing user IDs in a globally ordered table can hotspot a small number of users; salt or region partitioning can spread write load, at the cost of fan-out reads. Avoid large session blobs in a single HBase cell.

HBase execution

The local standalone HBase 2.6.6 service and Thrift gateway ran on ports 16010 and 9090. The loader processed 10,000 sessions for 1,000 users and wrote 99,873 page-view rows to user_events; a subsequent table scan returned the same 99,873 rows. The loader stores session metadata in family s and event attributes in family e. It also loaded 24,881 product/day rows. The user-prefix query for user_000042 returned 20 event rows, confirming the reverse-time key scan against a concrete user.

Comparison

MongoDB suits flexible, bounded business entities and multi-field lookups. HBase suits sparse, append-heavy event records and predictable row-key scans. Spark is used for broad joins and reductions rather than point retrieval. The databases complement one another; duplicating every record in both stores would add operational cost without improving the stated access patterns.

3. Spark processing and integration

The PySpark application reads multi-line JSON arrays, deduplicates primary IDs, parses event timestamps, explodes fulfilled transaction line items, and uses DataFrame joins and SQL. The co-purchase job self-joins distinct product IDs per order and counts order co-occurrence; a minimum support of two suppresses singletons. Cohort spend groups users by registration month and purchase month. Spark SQL joins line items to product categories and calculates revenue, units and order count.

Integrated business question

Which customer groups combine higher spending with stronger engagement, and where do sessions leave the purchase funnel? Spark joins user profiles, session counts and average durations, and fulfilled order totals. It calculates revenue per session and bands users into frequent (5+ orders), repeat (2-4), one-time (1), and no-purchase groups. The implemented store mode reads MongoDB profiles and fulfilled transaction aggregates and scans HBase user events directly through their native Python clients. It aggregates engagement by user, stages the extracts for Spark, joins on user_id, and writes metrics and customer segments. The verified run joined 1,000 profiles with 10,000 HBase sessions; it is distinct from the JSON-only batch path. CLV is represented by observed cumulative revenue, not a forecast.

Assignment-derived findings

Measure	Observed in assignment-derived run	Interpretation
Sessions and transactions	10,000 sessions; 2,500 transactions	Generated assignment data at reduced scale.
Funnel stages	9,885 product-view sessions; 6,732 cart sessions; 2,057 converted	20.6% of sessions converted; 30.6% of cart sessions converted.
Purchase frequency	77 frequent, 560 repeat, 248 one-time, 115 no purchase	Purchase-frequency bands use fulfilled transaction history.
Revenue by category	cat_009 leads at 274,758.91; cat_012 next at 191,715.93	Synthetic rankings are useful for workflow demonstration, not market inference.

MongoDB ranked prod_00292 first by units, with 40 units across 19 orders and 12,685.20 in item revenue. MongoDB and Spark independently ranked cat_009 first at 274,758.91 item revenue. The fulfilled orders produced 3,472 distinct co-purchase pairs before the minimum support filter; the most frequent pair appeared in three orders. These are synthetic reduced-scale results, not validated recommendations. Full-scale Spark summaries processed 2,000,000 sessions and 500,000 transactions: 405,887 converted sessions (20.3%), 439,067 fulfilled transactions, and 399,384,245.22 in fulfilled revenue across 25 category rows. These full-scale measures are separate from the sample-based MongoDB/HBase integration.

4. Visual findings

All figures below are generated by spark_analytics.py from the assignment generator code at the reduced local scale described above.

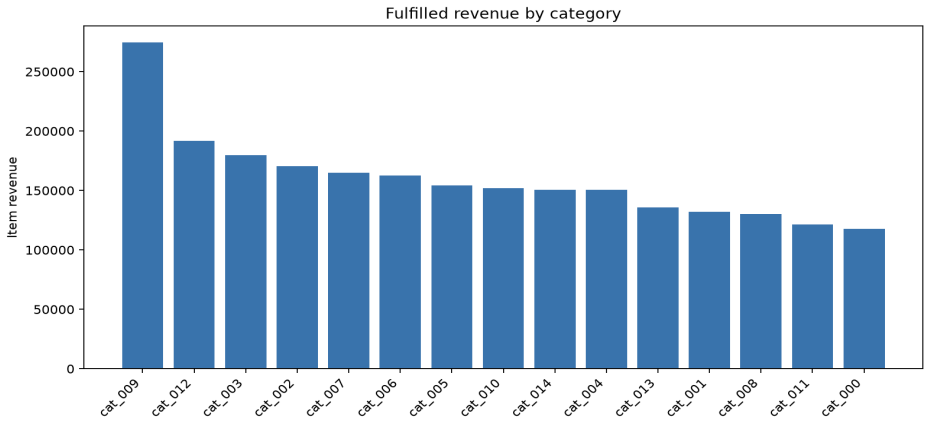


Figure 1. Fulfilled line-item revenue by category.

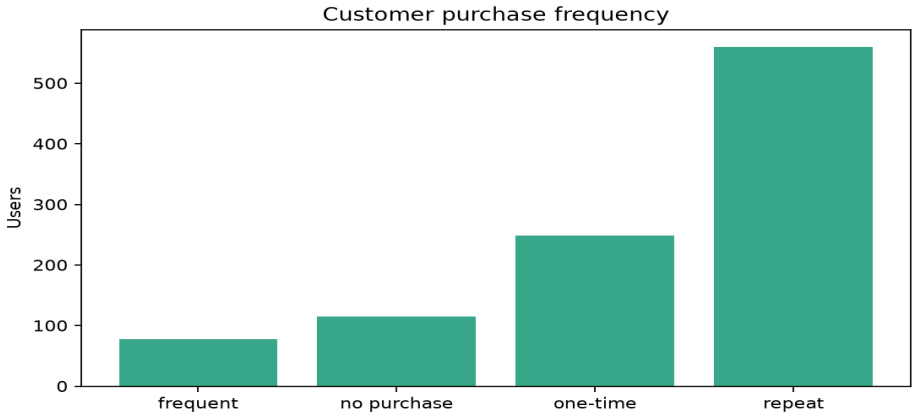


Figure 2. Customers grouped by purchase frequency.

4. Visual findings (continued)

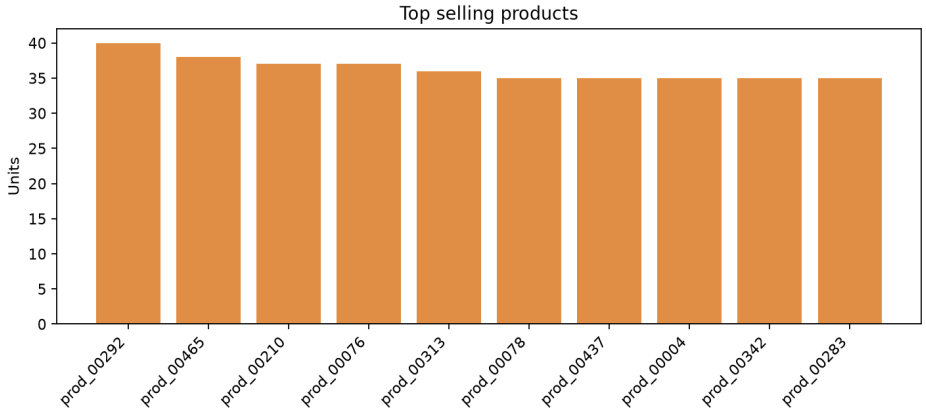


Figure 3. Top products ranked by units sold.

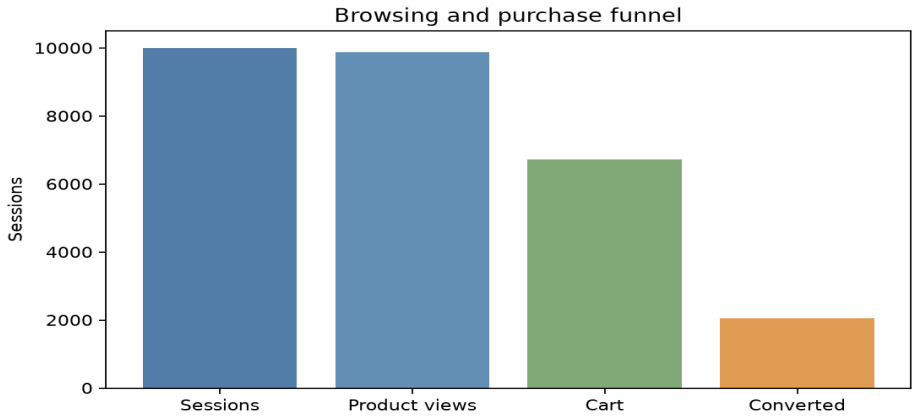


Figure 4. Sessions progressing through product view, cart and conversion.

Simulated streaming

A separate Spark Structured Streaming job uses the rate source to generate synthetic page events, groups them into five-second event-time windows by page type and category, and writes each microbatch result. The verified 40-second run produced 10 microbatches and 346 windowed groups. This is a simulated stream, not a live production event source.

5. Scalability, limitations and reproduction

Scalability considerations

At full scale, use bulk writes, bounded batches and retry-safe identifiers for MongoDB; shard only after choosing shard keys aligned to common filters. HBase event keys should distribute writes while retaining user-level scan locality. Spark should partition large input by date or stable key, avoid collecting unbounded data to the driver, broadcast only genuinely small dimensions, and persist only reused intermediate frames. Parquet outputs reduce repeated JSON parsing. Track skew, shuffle volume, memory pressure and storage growth before tuning partitions.

Limitations

The full-scale JSON dataset and focused Spark summaries were run at the assignment generator defaults. The full-scale summary reports 2,000,000 sessions, 405,887 converted sessions, 439,067 fulfilled transactions and 399,384,245.22 in fulfilled revenue. MongoDB and HBase ingestion and the direct cross-store join use the smaller representative sample. This local run is not a distributed-cluster performance benchmark. The synthetic generator has no age or demographic age field, so purchase-frequency and registration cohorts replace age segmentation. Session-linked and standalone transactions should be distinguished when calculating funnel attribution. Current inventory values are snapshots rather than a historical stock ledger. MongoDB loading and both aggregation pipelines were executed locally. The HBase event and product/day loads were executed through local Thrift; 99,873 event rows and 24,881 product/day rows were verified, and a 20-row scan was run for user_000042. The direct Spark integration was executed against MongoDB and HBase. The streaming example was exercised with synthetic Spark rate source. The original notebook JSON generator remains unmodified.

Reproduction

Install the packages in requirements.txt and start local HBase with the project script. Generate assignment-derived data with `python run_assignment_generator.py --out data_assignment --sessions 10000 --transactions 2500`. To regenerate full data use `python run_assignment_generator.py --out data_full --users 10000 --products 5000 --categories 25 --sessions 2000000 --transactions 500000`. Run full-scale summaries with `spark-submit --master local[2] spark_full_scale_summary.py --data data_full --schema-sample data_assignment --out output_full_scale_summary`. For a quick smoke run use `python generate_sample_data.py --out data`. On Windows set `$env:PYSPARK_PYTHON=(Get-Command python).Source`, then run `spark-submit --master local[*] spark_analytics.py --data data_assignment --out output`. Run the direct MongoDB/HBase integration with `spark-submit --master local[2] spark_analytics.py --source-mode stores --out output`, and simulated streaming with `spark-submit --master local[2] spark_streaming_simulation.py --duration-seconds 40 --rows-per-second 10`. Create this PDF with `python build_report.py`. MongoDB loader/queries run with `python mongodb_analytics.py --data data_assignment`.

Conclusion

The project demonstrates MongoDB for bounded business entities, HBase for time-ordered sparse events and product/day metrics, and Spark for batch, cross-store and simulated stream processing. Full-scale Spark batch analysis was run on the generator defaults; the verified database loads and direct store join use the assignment-scale sample. The generator remains unchanged, with bounded JSON chunking handled by its runner. Synthetic findings demonstrate the workflow and are not market estimates.